

What goes into context, and how

Atlas holds what is true; three hooks put the right slice of it in front of the agent at the moment it's needed — and keep the rest out. **The reads are compiled and injected, never trusted to happen.** This is the full lifecycle, for a component seat.

SESSION BEGINS — OR IS REBUILT

- Briefing compiled & injected `SessionStart`

EACH WORK TURN

- Writes screened before they land `PreToolUse`

SESSION ENDS

- Alignment & publish checked `Stop`

🔄 **Compaction, resume and `/clear` re-fire `SessionStart` mid-session** — there is no separate compaction hook. That re-fire, carrying `source: compact`, is the reorientation path (§03).

01 Three hooks on the session lifecycle

Every seat's `.claude/settings.json` wires exactly these. A hook is a shell script Claude Code runs at a lifecycle moment; what it prints and the code it exits with are how Atlas reaches into the session.

↓

SessionStart

fires on **startup · resume · `/clear` · compaction** — no matcher, so all four

🔄 RE-FIRES MID-SESSION

`atlas-context.sh`

Compiles the session's whole reading list into one file and **prints it to stdout** — **which Claude Code adds verbatim to the context window**. This is the main injection — and the **only** one: it also serves compaction and resume.

- 1 Read the hook payload from stdin (2 s-bounded) for its `source` — startup, resume, clear or compact.
- 2 Discover every wired sibling repo in the launch dir → **one briefing per seat**, not per repo.
- 3 Run `atlas-sync.sh`: resolve the method pin to its exact tag, apply branch policy, self-drift check.
- 4 Compile the briefing with `atlas_validate.py --emit-context` from a worktree of `origin/<work>` — always the work branch, never whatever's checked out.
- 5 Deliver raw contract artifacts (OpenAPI, schemas) to `ATLAS-CONTEXT.d/` as files — **referenced by path, not inlined**.
- 6 Record the compiled vault SHA (the Stop gate reads it) and print the byte/token size to stderr.

→ **INTO CONTEXT** the full **ATLAS-CONTEXT.md** briefing (see §02), with a provenance line naming the branch and commit it was built from.

Compaction & reorientation live here, not in a hook of their own. When the window is summarised, this hook re-fires with `source: compact`, rebuilds the briefing from scratch, and prepends the REORIENT directive (§03). Atlas rebuilds rather than trying to survive the summary.

PreToolUse before every **Write · Edit · MultiEdit · NotebookEdit**

`atlas-guard-write.sh`

Doesn't inject — it **governs what the agent may write into the vault**, deciding per file before the edit lands. Golden rule 2, made mechanical.

ALLOW

Own outbox components/<slug>/**, an additive architecture/proposals/*, or edges naming you in io-graph.yml. A both-hats seat also gets architecture/**.

DENY

Anything else in the vault — returned with a reason that **names the remedy** (file it in docs/needs/, or open a proposal). Not an

obstacle to route around.

CLOSED

A payload the guard **can't parse is denied**, never waved through. Guards fail closed.

Mechanism: emits a JSON `permissionDecision`; the write never reaches the tool unless allowed.



Stop at the **end of every turn**, before the agent hands back

`atlas-guard-publish.sh`

Two checks that can **refuse to let the turn end** — exit 2 blocks the stop and prints its message to `stderr`, which the agent sees and acts on.

GATE

Alignment. One `ls-remote` of the vault work branch (30 s throttle). Moved past the SHA your briefing was built from? Re-brief, reconcile, then finish — so an upstream change reaches a running seat by the end of its current turn.

NAG

Publish. Vault clone left dirty? You haven't run the outbox half of the protocol — `/atlas-publish`. Once per session.

OPEN

Fails open. Offline, no throttle window, nothing to publish → exit 0, the turn ends. Honest work is never trapped.

Mechanism: exit 0 lets the turn end; exit 2 blocks it and surfaces the reason. A once-per-turn flag prevents loops.

02 Anatomy of the injected briefing

What `SessionStart` actually puts in the window, in emit order. It opens with the development cycle and the retrieval invariant, then the sections below. The point of

the design is thrift: the things that bind everyone appear **once**, and anything a task can fetch on demand is an **index line**, not a body.

ATLAS - CONTEXT .md

compiled · one per seat · size reported every emit

§	Header + the development cycle reading-list rules · review architecture → examine edges → develop → test → publish	ALWAYS
1	Constitution the project's inviolable principles	FULL TEXT
2	Architecture in force accepted ADRs + standalone design docs	INDEX · READ ON DEMAND
3	Reference library provider-homed know-how delivered here	INDEX · READ ON DEMAND
4	Inputs — contracts delivered from other vaults cross-vault contracts, routed by consumer	FULL · YOURS ONLY
5	Needs addressed to several of my components shown once for the whole seat	SHARED ONCE
6	Per component — inputs I build against upstream contracts at their pinned version + artifact table	FULL TEXT
7	Per component — needs addressed to me consumers' open asks; answered ones collapse to one line	OPEN ONLY
8	In-flight proposals affecting me design decisions still being argued	FULL TEXT
9	Drift summary every edge: pinned vs latest	TABLE

03 Reorientation is a source, not a separate hook

"At what point" has four answers, because `SessionStart` carries a source — and the same hook serves all of them. A fresh start reads its orientation anyway; the other three are moments where the agent's working memory was rebuilt or lost, so the briefing arrives with an imperative to act on it, not skim past it. **Compaction is the reason the hook has no matcher.**

startup

A brand-new session. The briefing is injected plainly; the agent reads its standing orders (`AGENTS.md`) alongside.

compact

Context was summarised to fit. The conversation that held "who I am, what I was doing" is gone — the durable orientation is the briefing, so it leads with a directive.

resume

A session picked back up. Same risk as compaction: reconcile against the freshly-compiled briefing before acting.

/clear

The window was deliberately wiped. The hook re-injects the whole reading list, so a cleared session is never a blank one.

PREPENDED ON COMPACT · RESUME · CLEAR

🔄 **REORIENT — session was compacted.** Your working context was just rebuilt. Before your next action: read the briefing below in full, confirm which component you are and what you were doing, and resume from it. This is your complete orientation — do **not** ask the operator to re-orient you.

04 Kept out of context, on purpose

Managing a window is as much exclusion as injection. Atlas spends context only where it changes what the agent builds; everything else is reachable but not resident.

The wider vault

The retrieval invariant: a session reads its briefing, never browses. If the briefing is insufficient, that's a defect in `io-graph.yml` — fix the graph, don't widen the read.

Raw contract artifacts

OpenAPI specs and schemas land in `ATLAS-CONTEXT.d/` as byte-faithful files. The briefing points at them; a generator reads them by path. The window stays bounded.

Other components' deliveries

A contract delivered to a sibling gets you **one index line**, not its body. Consumer-routed — the vault-wide default once cost a seat 71% of its briefing.

Discharged & retired asks

An answered need collapses to a single line; a retired one (~~resolved~~, closed, done) drops out entirely. A briefing carries current obligations, not history.

05 Component seat vs arch seat

The arch seat has no slug and works the vault directly, so its hooks differ — same lifecycle, different payload.

COMPONENT SEAT

SessionStart → `atlas-context.sh` (the briefing above). **PreToolUse** → the write guard.

Stop → alignment + publish. Three hooks: it reads a compiled slice, and its writes are fenced to its outbox.

ARCH SEAT

SessionStart → `atlas-arch-context.sh` (`--emit-arch-context: constitution, architecture in force, estate & drift, review queue, next-steps, the bridge`). **Stop** → the arch alignment gate. No write guard — it authors the vault as reviewer.

Atlas method · `git` is the substrate · hooks make the protocol mechanical, not trust-based